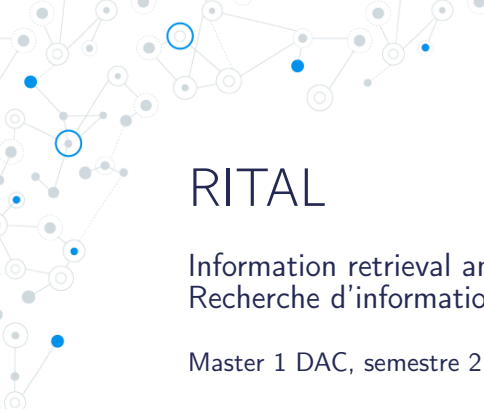




RITAL

Information retrieval and natural language processing
Recherche d'information et traitement automatique de la langue

Master 1 DAC, semestre 2

Nicolas Thome



1 Sequence Processing

2 Word Embeddings

Sequence Processing

- BoW and unsupervised methods (LSA/LDA): no explicit representation of words' sequences
- An important feature for many NLP task

⇒ How to include such sequential nature?

- Markov Models (MM) and HMM: generative models
- Conditional Random Fields (CRF): discriminative models → today
- RNNs & transformers: next course

- Sequence $\mathbf{x} := (x_1, \dots, x_T)$, $x_i \in Q = (q_1, \dots, q_N)$, λ MM parameters

$$\begin{aligned} p(\mathbf{x}|\lambda) &= p(x_1, \dots, x_T|\lambda) \\ &= \prod_{t=2}^T p(x_t|x_1, \dots, x_{t-1}, \lambda) p(x_1|\lambda) = \prod_{t=2}^T p(x_t|x_{t-1}, \lambda) p(x_1|\lambda) \\ &= \pi_{x_1} \prod_{t=2}^T a_{x_{t-1}, x_t} \end{aligned}$$

- $\lambda = (\Pi, A)$
 - $\Pi \in \mathbb{R}^N$: initial state probabilities
 - $A \in \mathbb{R}^{N \times N}$: transition probability matrix

- Sequence $\mathbf{x} := (x_1, \dots, x_T)$, $x_i \in Q = (q_1, \dots, q_N)$
- Hidden states $\mathbf{s} := (s_1, \dots, s_T)$, $s_i \in B = (b_1, \dots, b_M)$
- HMM parameters: $\lambda = (\boldsymbol{\Pi}, \mathbf{A}, \mathbf{B})$
 - $\Pi_i = p(s_1 = b_i)$, $\boldsymbol{\Pi} \in \mathbb{R}^N$
 - $a_{ij} := p(s_t = b_j | s_{t-1} = b_i)$ $\mathbf{A} \in \mathbb{R}^{N \times N}$
 - $b_j(i) := p(x_t = q_i | s_t = b_j)$ $\mathbf{B} \in \mathbb{R}^{M \times N}$
- Probability of state $\mathbf{s} = (s_1, \dots, s_T)$ and observation $\mathbf{x} = (x_1, \dots, x_T)$ sequences :

$$p(\mathbf{s}, \mathbf{x} | \lambda) = \pi_{s_1} \prod_{t=2}^T a_{s_{t-1}, s_t} b_{s_t}(x_t)$$

- Probability of state $\mathbf{s} = (s_1, \dots, s_T)$ and observation $\mathbf{x} = (x_1, \dots, x_T)$ sequences :

$$p(\mathbf{s}, \mathbf{x} | \boldsymbol{\lambda}) = \pi_{s_1} \prod_{t=2}^T a_{s_{t-1}, s_t} b_{s_t}(x_t)$$

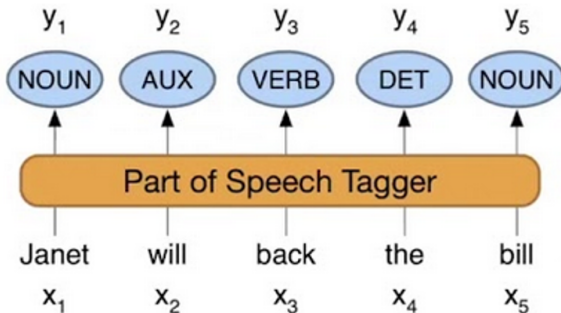
- If $\mathbf{x}$ and $\mathbf{s}$ observed (no hidden states), maximum likelihood estimation easy:

- $\Pi_i = \frac{N_i}{K}$, $N_i \# s_1 = q_i$
- $a_{ij} = \frac{n_{ij}}{n_i}$, $n_{ij} \#$ transitions $b_i \rightarrow b_j$, $n_i \#$ state q_i
- $b_j(i) = \frac{n'_{ij}}{n'_j}$, $n'_j \#$ occurrences $(x_t = q_i, s_t = b_j)$, $n'_j \#$ state b_j

$\Rightarrow$ counting!

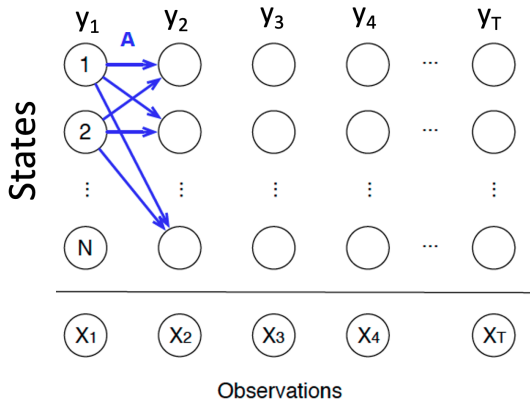
Sequences: ex for Part of Speech tagging

- Context: labelling of each word $\rightarrow$ Part of Speech (PoS)
- Input $\mathbf{x} = (x_1, \dots, x_T)$ a sentence with T words
- Output: $\mathbf{y} = (y_1, \dots, y_T)$ a sequence of T PoS

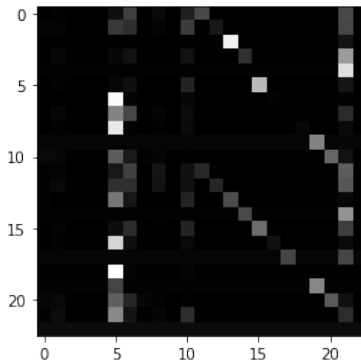


HMMs

- Input $\mathbf{x} = (x_1, \dots, x_T)$ a sentence with T words
 - M words, e.g. $M \sim 20000$
- Output: $\mathbf{y} = (y_1, \dots, y_T)$ a sequence of T PoS
 - N PoS tags, e.g. $N = 20$
- HMM parameters: $\boldsymbol{\Pi} \in \mathbb{R}^N$, $\mathbf{B} \in \mathbb{R}^{M \times N}$,
transition matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$
- Training: states observed (not hidden)
 - $\boldsymbol{\Pi}$, $\mathbf{A}$, $\mathbf{B}$ counting!
- Predicting PoS tags for a new sequence
 - Viterbi (see MAPSI)



- Some words are ambiguous: one word $\rightarrow$ several PoS tags
- Sequence order: help to resolve some ambiguities



General framework

- Input $\mathbf{x} \in \mathcal{X}$: arbitrary
- Output $y \in \mathcal{Y}$: discrete output space
 - $\mathbf{y} \in \mathcal{Y}$ can be anything: vector, sequence (of vectors), graph, etc
- Assumption, there exist a function $\psi(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^d$
- CRF: model of conditional probability function:

$$P(\mathbf{y}|\mathbf{x}, \mathbf{w}) = \frac{e^{\mathbf{w}^T \psi(\mathbf{x}, \mathbf{y})}}{\sum_{\mathbf{y}' \in \mathcal{Y}} e^{\mathbf{w}^T \psi(\mathbf{x}, \mathbf{y}')}}}$$

- $Z(\mathbf{x}) = \sum_{\mathbf{y}' \in \mathcal{Y}} e^{\mathbf{w}^T \psi(\mathbf{x}, \mathbf{y}')}:$ partition function

General framework for training

- A set of N training pairs $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i \in \{1; N\}}$
- i.i.d samples, maximizing conditional log likelihood $\mathcal{L}(\mathbf{w}) = \sum_{i=1}^N \log [P(\mathbf{y}_i | \mathbf{x}_i, \mathbf{w})]$

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^N \mathbf{w}^T \psi(\mathbf{x}_i, \mathbf{y}_i) - \log \left(\sum_{\mathbf{y}' \in \mathcal{Y}} e^{\mathbf{w}^T \psi(\mathbf{x}_i, \mathbf{y}')} \right) = \mathbf{w}^T \psi(\mathbf{x}_i, \mathbf{y}_i) - \log (Z(\mathbf{x}_i)) \quad (1)$$

- $\mathbf{y}_i$ GT output for input $\mathbf{x}_i$
- Eq (1) convex, can be solved with gradient descent
 - Can add ℓ_2 regularization $\|\mathbf{w}\|^2$ on Eq (1) by putting Gaussian prior $P(\mathbf{w}) \sim \mathcal{N}(0, \sigma^2)$

Classification

- We need to define $\mathcal{X}, \mathcal{Y}, \psi(\mathbf{x}, \mathbf{y})$
- Ex: $\mathcal{X} = \mathbb{R}^d$, $\mathcal{Y} = \{1; K\}$ (K number of classes), and $\psi(\mathbf{x}, \mathbf{y}) \in \mathbb{R}^{d \cdot K}$

$$\psi(\mathbf{x}, \mathbf{y}) = (\delta_{y,1}\mathbf{x}, \dots, \delta_{y,k}\mathbf{x}, \dots, \delta_{y,K}\mathbf{x})$$

- $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_k, \dots, \mathbf{w}_K)$, where $\mathbf{w}_k \in \mathbb{R}^d \rightarrow \mathbf{w}^T \psi(\mathbf{x}, k) = \mathbf{w}_k^T \mathbf{x}$
- Training: $\{(\mathbf{x}_i, k_i^*)\}_{i \in \{1; N\}}$

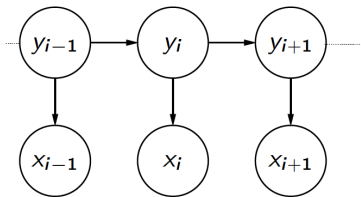
$$\Rightarrow P(k|\mathbf{x}, \mathbf{w}) = \frac{e^{\mathbf{w}_k^T \mathbf{x}}}{\sum_{k'=1}^K e^{\mathbf{w}_{k'}^T \mathbf{x}}}, \text{ and } \mathcal{L}(\mathbf{w}) = \sum_{i=1}^N \mathbf{w}_{k_i^*}^T \mathbf{x}_i - \log \left(\sum_{k=1}^K e^{\mathbf{w}_k^T \mathbf{x}_i} \right)$$

Logistic Regression

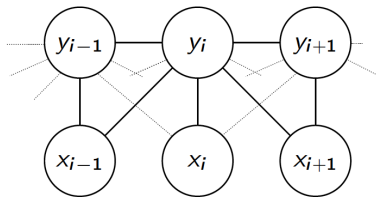
Factorization of cond. prob. $P(\mathbf{y}|\mathbf{x})$ in linear-chain CRF:

$$\mathbf{w}^T \psi(\mathbf{x}, \mathbf{y}) = \sum_{t=1}^T \sum_{k=1}^K F_k(y_{t-1}, y_t, \mathbf{x}) = \sum_{t=1}^T \sum_{k=1}^K [u_k(y_t, \mathbf{x}) + p_k(y_{t-1}, y_t, \mathbf{x})]$$

- $u_k(y_t, \mathbf{x})$: unary term $\sim$ observation matrix B in HMM BUT $u_k(y_t, \mathbf{x}_t)$ in HMM
- $p_k(y_{t-1}, y_t, \mathbf{x})$: pairwise terms with Markovian assumption $\sim$ transition matrix A in HMM BUT $p_k(y_{t-1}, y_t)$ in HMM



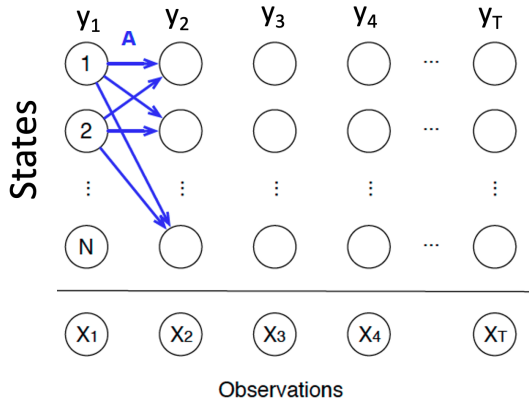
HMM



CRF

CRFs $\succ$ HMMs because:

- Discriminative model for identifying PoS tags vs generative model for HMMs
- More flexible conditional independence assumption
- Stronger features



Examples of features for PoS Tagging

```
def features_full(sentence, index):  
    return {  
        'word': sentence[index],  
        'is_first': index == 0,  
        'is_last': index == len(sentence) - 1,  
        'is_capitalized': sentence[index][0].upper() == sentence[index][0],  
        'is_all_caps': sentence[index].upper() == sentence[index],  
        'is_all_lower': sentence[index].lower() == sentence[index],  
        'prefix-1': sentence[index][0],  
        'prefix-2': sentence[index][:2],  
        'prefix-3': sentence[index][:3],  
        'suffix-1': sentence[index][-1],  
        'suffix-2': sentence[index][-2:],  
        'suffix-3': sentence[index][-3:],  
        'prev_word': 'if index == 0 else sentence[index - 1],  
        'next_word': 'if index == len(sentence) - 1 else sentence[index + 1],  
        'has_hyphen': '-' in sentence[index],  
        'is_numeric': sentence[index].isdigit(),  
        'capitals_inside': sentence[index][1:].lower() != sentence[index][1:]  
    }
```


$$P(\mathbf{y}|\mathbf{x}) = \frac{e^{\sum_{t=1}^T \sum_{k=1}^K [\theta_k u_k(y_t, \mathbf{x}) + \lambda_k p_k(y_{t-1}, y_t, \mathbf{x})]}}{Z(\mathbf{x})}$$
$$Z(\mathbf{x}) = \sum_{\mathbf{y}' \in \mathcal{Y}} e^{\sum_{t=1}^T \sum_{k=1}^K [\theta_k u_k(y'_t, \mathbf{x}) + \lambda_k p_k(y'_{t-1}, y'_t, \mathbf{x})]} \quad (2)$$

- Problem: $\mathcal{Y}$ huge: $\dim N^T$
- Model prediction: $\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} P(\mathbf{y}|\mathbf{x})$
- Training : computing normalization factor $Z(\mathbf{x}) = \sum_{\mathbf{y}' \in \mathcal{Y}} \dots$
- Brute-force computation intractable, must exploit structure...

$\Rightarrow$ dynamic programming

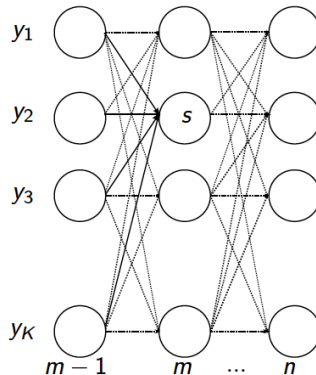
Inference

- Model prediction:

$$\begin{aligned}
 \hat{\mathbf{y}} &= \arg \max_{\mathbf{y}} P(\mathbf{y}|\mathbf{x}) \\
 &= \arg \max_{\mathbf{y}} \sum_{t=1}^T \sum_{k=1}^K F_k(y_{t-1}, y_t, \mathbf{x}) \\
 &:= \arg \max_{\mathbf{y}} \sum_{t=1}^T g_t(y_{t-1}, y_t)
 \end{aligned}$$

- Viterbi algorithm $\sim$ HMM

$$\delta_m(s) \triangleq \max_{\{y_1, \dots, y_{m-1}\}} \left[\sum_{i=1}^{m-1} g_i(y_{i-1}, y_i) + g_m(y_{m-1}, s) \right]$$



Training

$$\mathcal{L}(\mathbf{w}) = \mathbf{w}^T \psi(\mathbf{x}_i, \mathbf{y}_i) - \log(Z(\mathbf{x}_i))$$

- $Z(\mathbf{x}_i)$ intractable
- Forward-backward algorithm

Algorithm 1 The forward-backward algorithm for the probability calculation of the CRF

Input: The model $P(Y|X)$, the input sequence $\mathbf{x}$, the output sequence $\mathbf{y}$, and the location i

Output: The conditional probabilities $P(Y_i = y_i | \mathbf{x})$, $P(Y_{i-1} = y_{i-1}, Y_i = y_i | \mathbf{x})$

1. Let $M_i(y_{i-1}, y_i | \mathbf{x}) = \exp(\sum_{k,k} \lambda_k t_k(y_{i-1}, y_i, \mathbf{x}, i) + \sum_{j,j} \mu_j s_j(y_i, \mathbf{x}, i))$, $y_0 = start$, $y_{n+1} = stop$

2. Initialization, $\alpha_0(y_0 | \mathbf{x}) = 1$, $\beta_{n+1}(y_{n+1} | \mathbf{x}) = 1$

3. Recursion, for $k = 1, 2, \dots, i$

$$\alpha_k^T(y_k | \mathbf{x}) = \alpha_{k-1}^T(y_{k-1} | \mathbf{x}) M(y_{k-1}, y_k | \mathbf{x})$$

4. Recursion, for $j = n, n-1, \dots, i+1, i, i-1, \dots, 1$

$$\beta_j(y_j | \mathbf{x}) = M_{j+1}(y_j, y_{j+1} | \mathbf{x}) \beta_{j+1}(y_{j+1} | \mathbf{x})$$

5. Calculation, $Z(\mathbf{x}) = \mathbf{1}^T \times \beta_1(\mathbf{x})$

6. Calculation, $P(Y_i = y_i | \mathbf{x}) = \alpha_i^T(y_i | \mathbf{x}) \beta_i(y_i | \mathbf{x}) / Z(\mathbf{x})$

$$P(Y_{i-1} = y_{i-1}, Y_i = y_i | \mathbf{x}) = \alpha_{i-1}^T(y_{i-1} | \mathbf{x}) M_i(y_{i-1}, y_i | \mathbf{x}) \beta_i(y_i | \mathbf{x}) / Z(\mathbf{x})$$

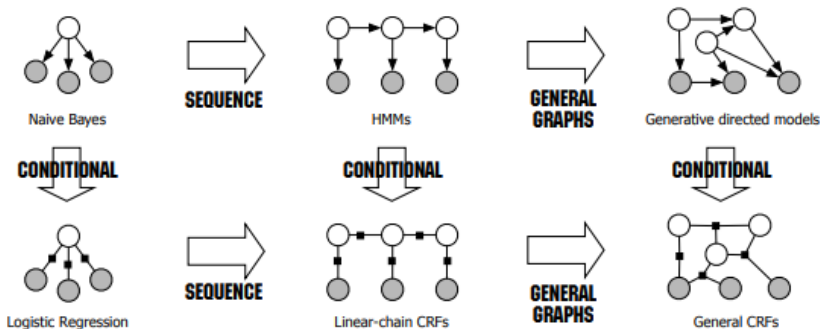


Fig. 2.3 Diagram of the relationship between naive Bayes, logistic regression, HMMs, linear-chain CRFs, generative models, and general CRFs.

Word Embeddings

1 Sequence Processing

2 Word Embeddings

- At the **document** scale:
 - ⇒ Several efficient algorithm to classify/categorize
- At the **sentence** scale:
 - Segmentation
 - POS, NER, SRL...
 - ⇒ Sequential approaches (HMM, CRF)
- At the **word** scale:
 - Defining / understanding the word
 - Linking the word with antonym, synonym, etc, ...
 - Binding the word to a lexical field
 - ⇒ Latent semantics / linguistic resources

⇒ How deep learning / representation learning can improve our previous solutions?

- Simplest encoding of text inputs: **one-hot representation**
- Binary vector of vocabulary size $|V|$, with 1 corresponding to term index
- $|V|$ small for chars (~ 10), large for words ($\sim 10^4$), huge for sentences
- Basis for constructing Bag of Word (BoW) Models

the dog is on the table

0	0	1	1	0	1	1	1
are	cat	dog	is	now	on	table	the

- Handcrafted feature used with ML shallow models, e.g. kernels methods
- Still very competitive for some NLP tasks, e.g. text topic classification
- Can be extended to (bags of) bi-grams for e.g. language identification

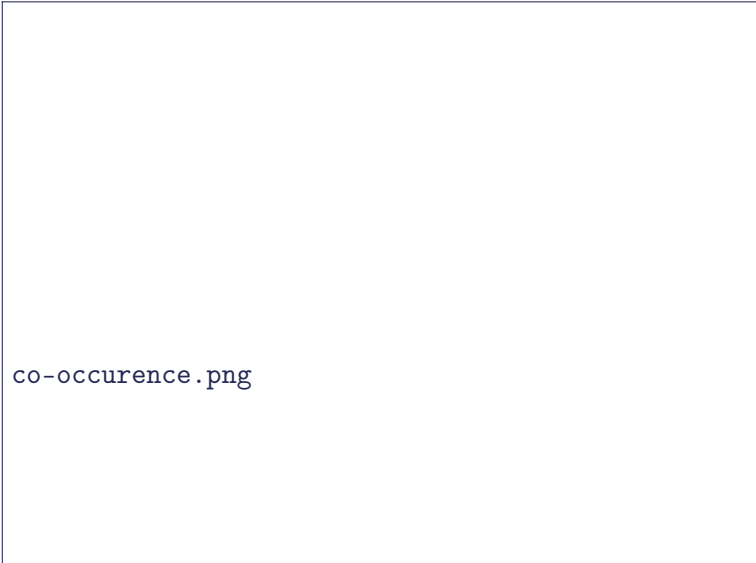
- Limitation: $\langle r(\text{"motel"}) ; r(\text{"hotel"}) \rangle = 0$

one-hot-limits.png

- Learn mapping from one-hot encoding to a smaller vectorial space
- General idea: representing a word by means of its neighbors

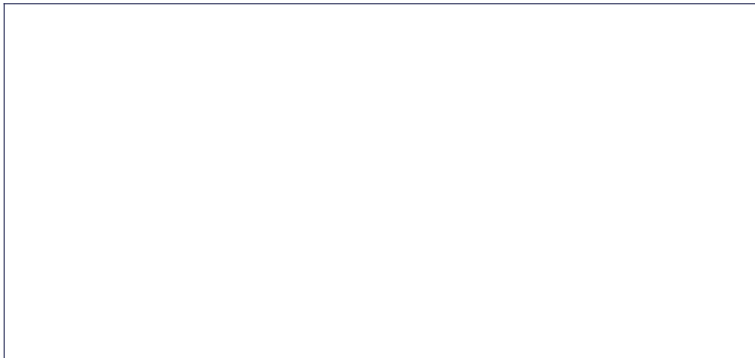
context-embeddings.png

- Simplest historical strategy to represent context: co-occurrence matrices

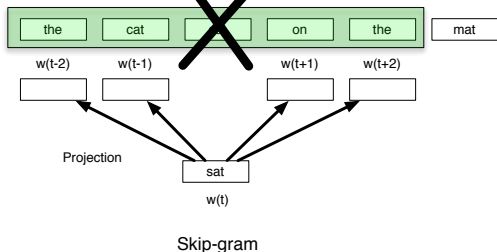


co-occurrence.png

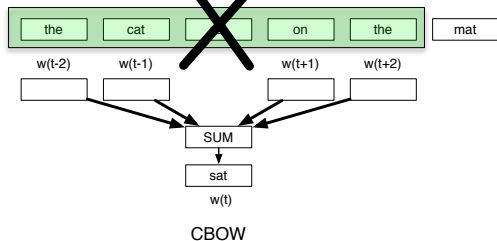
- Modern approaches: directly learn low-dim text vectors
- **Word2vec** [MSC⁺13]: similar words $\Leftrightarrow$ similar contexts
 - Predict surrounding words (context) from central word: **CBoW**
 - Predict context from central word: **Skipgram**
- N.B.: cast unsupervised task as supervised one: "auto-supervision" (more next course) $\neq$ reconstruction / ML, e.g. SVD



Sliding window (size = 5)



Sliding window (size = 5)



Skip-Gram (& *negative sampling* implementation) : easy learning

- Local analysis
- *predictive* criterion : estimating missing value
- Sliding window = local context C of word w



Mikolov, Sutskever, Chen, Corrado, Dean, NIPS 2013 (arXiv 2012)
Distributed representations of words and phrases and their compositionality

- From a word $\mathbf{w}$, one-hot encoded (size $|V|$) $\Rightarrow$ infer its context C
- Project $\mathbf{w}$ into latent space $\mathbf{h}$, size $N \Rightarrow$ matrix $\mathbf{W}$ (select j^{st} column)
- Project $\mathbf{h}$ back into $|V|$ space $\Rightarrow$ matrix $\mathbf{W}' + \text{soft-max}$
- Loss function: average cross-entropy for randomly context words



- From a context C one-hot encoded (size $|V|$) $\Rightarrow$ infer central word w
- Encode C into latent space h + average (or $\sim$ project avg $C \Leftrightarrow$ BoW)
- Decode h back into $|V|$ space + soft-max
- Loss function: cross-entropy for central word

CBoW-new.png

- Skipgram and CBoW trained with back-prop (see next)
- Soft-max : normalization over a huge vocabulary $|V|$:

$$\mathcal{L}(\mathbf{W}) = -\frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log \left(\frac{\exp(v'_{w_{t+j}} v_{w_t})}{\sum_i \exp(v'_{w_i} v_{w_t})} \right) \text{ Options:}$$

- Hierarchical soft-max
 - Use sigmoid instead + negative sampling (see next)
- CBoW works well for frequent words, Skipgram for rare words

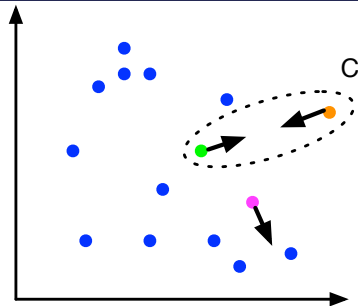
Conclusion

- Unsupervised: can be trained on huge generalist corpus
 - Transfer and fine-tuning possible on specific supervised tasks
 - Word2Vec and Glove $\Leftrightarrow$ VGG or ResNet for vision
 - BUT only one layer transfer
- Extension of Skipgram for sentences Skip-Thought Vectors [KZS⁺15]
 - predict the surroundings sentences of a given sentence
 - Extended to discriminative learning recently [LL18] $\Rightarrow$ faster training

- Given word w and local contexts C :

$$\text{Idée SG: } \arg \max_{\theta} \prod_C \prod_{w \in C} p(C|w; \theta)$$

- $p(D = 1|w_i, w_j; \theta) \Rightarrow$ proba. that w_i and w_j occur in the same context



$$\arg \max_{\theta} \prod_{i,j \in C} p(D = 1|w_i, w_j; \theta) + \underbrace{\prod_{i,j \in \bar{C}} p(D = 0|w_i, w_j; \theta)}_{\text{Negative Sampling}}$$



Goldberg, Levy, arXiv 2014

word2vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method



Hammer, NN 2002

Generalized Relevance Learning Vector Quantization

$$\arg \max_{\theta} \prod_{i,j \in C} p(D = 1 | w_i, w_j; \theta) + \underbrace{\prod_{i,j \in \bar{C}} p(D = 0 | w_i, w_j; \theta)}_{\text{Negative Sampling}}$$

■ Using logistic function : $p(D = 1 | w_i, w_j) = \frac{1}{1 + \exp(-z_i \cdot z_j)}$

■ Global log-likelihood : $\arg \max_Z \left(\sum_{i,j \in C} \log \sigma(z_i \cdot z_j) + \sum_{i,j \in \bar{C}} \log \sigma(-z_i \cdot z_j) \right)$

σ : fct sigmoïde, C : Set of Cooccurrences, $\bar{C}$: Set of Non-Cooc

■ Stochastic Gradient Descent + triplet loss

■ Frequent word subsampling trick : picking words with

$$p(w_i) = 1 - \sqrt{\frac{t}{\text{freq}(w_i)}}, \quad t = 10^{-5}$$

- Visualization of the latent space: t-SNE
- Semantic clustering of concept
- Ex with paragraph word2vec trained on wikipedia: see

<http://colah.github.io/posts/2015-01-Visualizing-Representations/>

embeddings-visu.png

- Word Vector Analogies: $a:b :: c:?$, e.g. $\text{man:woman}:: \text{king:?}$
 $\Rightarrow$ map the relation between a and b to c

- Assumption: can be done with simple algebraic operations (sum, subtraction): $r(c) + r(b) - r(a)$
- Disentangling in the learning representation space

$$d = \arg \max_i \frac{[r(c) + r(b) - r(a)]^T r(i)}{\|r(c) + r(b) - r(a)\|}$$

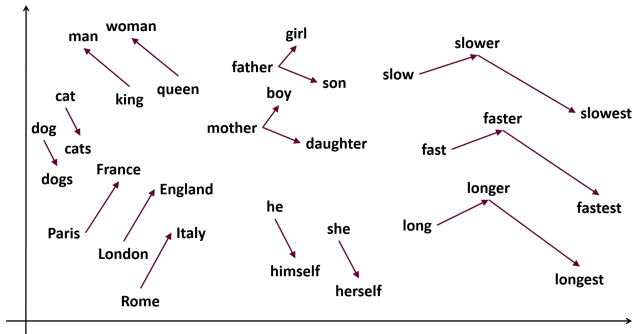
embeddings-analogy1.png

a is to b what c is to ???



$$z_b - z_a + z_c$$

Syntactical property (1):



$$z_{\text{woman}} - z_{\text{man}} \approx z_{\text{queen}} - z_{\text{king}}$$

$$z_{\text{kings}} - z_{\text{king}} \approx z_{\text{queens}} - z_{\text{kings}}$$

Query:

$$z_{\text{woman}} - z_{\text{man}} + z_{\text{king}} = z_{\text{req}}$$

Nearest neighbor:

$$\underset{i}{\operatorname{argmin}} \|z_{\text{req}} - z_i\| = \text{queen}$$

a is to *b* what *c* is to ???

$\Leftrightarrow$

$$\mathbf{z}_b - \mathbf{z}_a + \mathbf{z}_c$$

Syntactical property (2): Query:

$$\mathbf{z}_{easy} - \mathbf{z}_{easiest} + \mathbf{z}_{luckiest} = \mathbf{z}_{req}$$

Nearest neighbor:

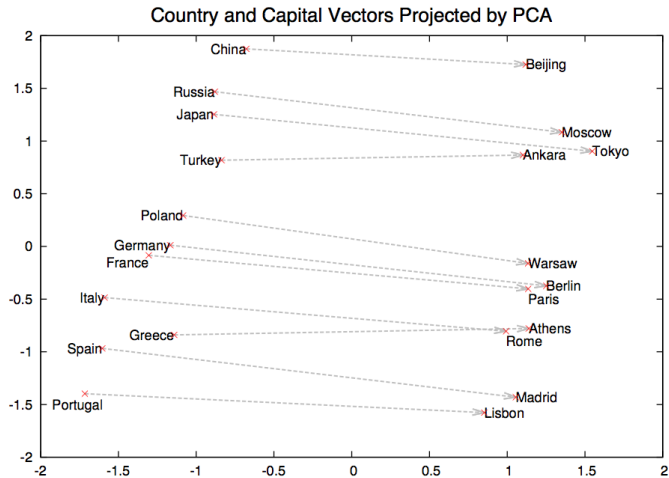
$$\underset{i}{\operatorname{argmin}} \|\mathbf{z}_{req} - \mathbf{z}_i\| = \text{lucky}$$

a is to *b* what *c* is to ???



$$z_b - z_a + z_c$$

Semantic Property (1):



a is to b what c is to ???

$\Leftrightarrow$

$$z_b - z_a + z_c$$

Semantic Property (2)

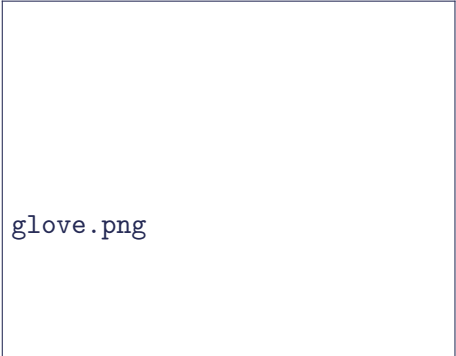
Czech + currency	Vietnam + capital	German + airlines	Russian + river	French + actress
koruna	Hanoi	airline Lufthansa	Moscow	Juliette Binoche
Check crown	Ho Chi Minh City	carrier Lufthansa	Volga River	Vanessa Paradis
Polish zolty	Viet Nam	flag carrier Lufthansa	upriver	Charlotte Gainsbourg
CTK	Vietnamese	Lufthansa	Russia	Cecile De

Table 5: Vector compositionality using element-wise addition. Four closest tokens to the sum of two vectors are shown, using the best Skip-gram model.

- W2V as input representation for words in PoS
- Using logistic regression as baseline
- Using W2V representation in CRF: W2V, W2V + CRF features

- Glove. Global vectors for word Representation
- Based on co-occurrence matrix $\mathbf{X} \Rightarrow$ leverage global context
 - Based on ratio of co-occurrence probabilities
- Explicitly enforcing semantic embedding, i.e. $\mathbf{w}_i^T \mathbf{w}_j \approx \log(X_{ij})$:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) (\mathbf{w}_i^T \mathbf{w}_j - \log(X_{ij}))$$



glove.png

- $\mathbf{w}_i$ main word embedding, $\mathbf{w}_j$ context embedding ($\mathbf{X}$ computed on local windows)
- $f(X_{ij}) = \begin{cases} (\frac{x}{x_{max}})^\alpha & \text{if } x \leq x_{max} \\ 1 & \text{otherwise} \end{cases}$

- Predicting instead of counting... **Not the good explanation!**

- Learning a **local semantics** !

⇒ GloVe $\approx$ **PLSA** + **local context** + **embedding** based implem.

- $X \in \mathbb{R}^{V \times V}$ word co-occurrence matrix

- X_{ij} frequency of word i co-occurring with word j

- $X_i = \sum_k X_{ik}$ total number of occurrences of word i in corpus

- $P_{ij} = P(j|i) = \frac{X_{ij}}{X_i}$ a.k.a. probability of word j occurring within the context of word i

- $w \in \mathbb{R}^z$ a word embedding of dimension z

- $\tilde{w} \in \mathbb{R}^z$ a context word embedding of dimension z



Baroni, Dinu & Kruszewski, ACL 2014

Don't count, predict! A systematic comparison of context-counting vs. context-predicting semantic vectors



Pennington, Socher & Manning, EMNLP 2014

Glove: Global Vectors for Word Representation

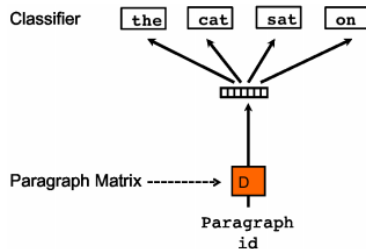
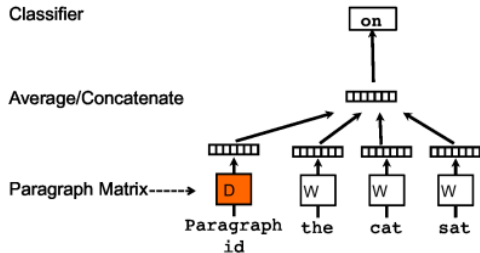
`glove-analogy1.png`

glove-analogy2.png

glove-analogy3.png

One remaining question: we learned a powerful semantics at the **word level**... How scaling to the **sentence** or **document level**?

One remaining question: we learned a powerful semantics at the **word level**... How scaling to the **sentence** or **document level**?



Q. Le, T. Mikolov, ICML 2014

Distributed representations of sentences and documents

One remaining question: we learned a powerful semantics at the **word level**... How scaling to the **sentence** or **document level**?

Or simple averaging of word embeddings:

- + great results on small word groups
- poor results on larger groups
 - quickly converge to a central abstract point of the latent space

One remaining question: we learned a powerful semantics at the **word level**... How scaling to the **sentence** or **document level**?

- 1 Aggregate multiple words associated to a single entity

- *Pointwise Mutual Information* threshold:

$$\text{score}(w_i, w_j) = \frac{\text{count}(w_i w_j) - \delta}{\text{count}(w_i) \times \text{count}(w_j)}.$$

- 2 Include new terms in the dictionary before running word2vec

Newspapers			
New York	New York Times	Baltimore	Baltimore Sun
San Jose	San Jose Mercury News	Cincinnati	Cincinnati Enquirer
NHL Teams			
Boston	Boston Bruins	Montreal	Montreal Canadiens
Phoenix	Phoenix Coyotes	Nashville	Nashville Predators
NBA Teams			
Detroit	Detroit Pistons	Toronto	Toronto Raptors
Oakland	Golden State Warriors	Memphis	Memphis Grizzlies
Airlines			
Austria	Austrian Airlines	Spain	Spainair
Belgium	Brussels Airlines	Greece	Aegean Airlines
Company executives			
Steve Ballmer	Microsoft	Larry Page	Google
Samuel J. Palmisano	IBM	Werner Vogels	Amazon

- FastText [JGBM17]
- ELMo [PNI⁺18]
 - Standard word embeddings (word2Vec, Glove), word embedding context independent, e.g. "stick"
 - ELMo: an embedding based on the context it's used in - to both capture the word meaning in that context as well as other contextual information
- BERT [DCLT18]: Bidirectional Encoder Representations from Transformers (next)



Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova.
BERT: pre-training of deep bidirectional transformers for language understanding.
CoRR, abs/1810.04805, 2018.



Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov.
Bag of tricks for efficient text classification.
In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 427–431. Association for Computational Linguistics, 2017.



Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler.
Skip-thought vectors.
In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 3294–3302. Curran Associates, Inc., 2015.



Lajanugen Logeswaran and Honglak Lee.
An efficient framework for learning sentence representations.
In *In ICLR*, 2018.



Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean.
Distributed representations of words and phrases and their compositionality.
In C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc., 2013.



Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer.
Deep contextualized word representations.
In Marilyn A. Walker, Heng Ji, and Amanda Stent, editors, *NAACL-HLT*, pages 2227–2237. Association for Computational Linguistics, 2018.



Jeffrey Pennington, Richard Socher, and Christopher D. Manning.

Glove: Global vectors for word representation.

In *EMNLP*, 2014.